accomplish specific user tasks. Design issues such as response time, command and action structure, error handling, and help facilities are considered as the design model is refined. A variety of implementation tools are used to build a prototype for evaluation by the user.

Like interface design for conventional software, the design mobile app interface describes the structure and organization of the user interface and includes a representation of screen layout, a definition of the modes of interaction, and a description of navigation mechanisms. A set of interface design principles and an interface design workflow guide the mobile app designer when layout and interface control mechanisms are designed.

The user interface is the window into the software. In many cases, the interface molds a user's perception of the quality of the system. If the "window" is smudged, wavy, or broken, the user may reject an otherwise powerful computer-based system. Usability and accessibility issues in the interface may also cause users to find an alternative product that better meets their needs and expectations.

PROBLEMS AND POINTS TO PONDER

12.1. Describe the worst interface that you have ever worked with and critique it relative to the concepts introduced in this chapter. Describe the best interface that you have ever worked with and critique it relative to the concepts introduced in this chapter.

12.2. Consider one of the following interactive applications (or an application assigned by your instructor):

a. A desktop publishing system
b. A computer-aided design system
c. An interior design system (as described in Section 12.4.2)
d. An automated course registration system for a university
e. A library management system
f. An Internet-based polling booth for public elections
g. A home banking system
h. An interactive application assigned by your instructor

Develop a user model, design model, mental model, and an implementation model, for any one of these systems.

12.3. Perform a detailed task analysis for any one of the systems listed in Problem 12.2.

12.4. Create a customer journey map for one of the systems listed in Problem 12.2.

12.5. Continuing Problem 12.2, define interface objects and actions for the application you have chosen. Identify each object type.

12.6. Develop a set of screen layouts and organize them into a storyboard for the system you chose in Problem 12.2.

12.7. Use a prototyping tool like Keynote to create an interactive prototype for the storyboard you created in Problem 12.6.

12.8. Describe your approach to user help facilities for the task analysis design model and task analysis you performed as part of Problems 12.3, 12.4, and 12.5.

12.9. Provide a few examples that illustrate why response time variability can be an issue.

12.10. Develop an approach that would automatically integrate error messages and a user help facility. That is, the system would automatically recognize the error type and provide a help window with suggestions for correcting it. Perform a reasonably complete software design that considers appropriate data structures and algorithms.

12.11. Develop an interface evaluation questionnaire that contains 20 generic questions that would apply to most interfaces. Have 10 classmates complete the questionnaire for an interactive system that you all use. Summarize the results, and report them to your class.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

13 DESIGN FOR MOBILITY

Mobile devices—smartphones, tablets, wearable devices, handheld gaming devices, and other specialized products—have become the new face of computing. According to Pew Research Center [Pew18], in the United States 77 percent of people own a smartphone and 50 percent of people own a tablet computer of some kind. Mobile computing has become a dominant force.

KEY CONCEPTS

aesthetic design 277
architecture design 278
challenges 265
cloud computing 273
component-level design 282
content architecture 279
content design 277
content objects 277
context-aware apps 274
design
  best practices 285
  mistakes 272
  pyramid 275
  quality 282
graphic design 277
mobile architectures 273
mobile development life cycle 268
model-view-controller 279
navigation design 280
quality checklist 285
technical considerations 266
user interface design 270
WebApp architecture 279

QUICK LOOK

What is it? Mobile design encompasses technical and nontechnical activities that include: establishing the look and feel of the mobile application (including mobile apps, WebApps, virtual reality, and games), creating the aesthetic layout of the user interface, establishing the rhythm of user interaction, defining the overall architectural structure, developing the content and functionality that reside within the architecture, and planning the navigation that occurs within the mobile product.

Who does it? Software engineers, graphic designers, content developers, security specialists, and other stakeholders all participate in the creation of a mobile design model.

Why is it important? Design allows you to create a model that can be assessed for quality and improved before content and code are generated, tests are conducted, and end users become involved in large numbers. Design is the place where mobile app quality is established.

What are the steps? Mobile design encompasses six major steps that are driven by information obtained during requirements modeling and are described in this chapter.

What is the work product? A design model that encompasses content, aesthetics, architecture, interface, navigation, and component-level design issues is the primary work product that is produced during mobile design.

How do I ensure that I've done it right? Each element of the design model is reviewed in an effort to uncover errors, inconsistencies, or omissions. In addition, alternative solutions are considered, and the degree to which the current design model will lead to effective implementation on a variety of software platforms and devices is also assessed.

In his authoritative book on Web design, Jakob Nielsen [Nie00] states: "There are essentially two basic approaches to design: the artistic ideal of expressing yourself and the engineering ideal of solving a problem for a customer." During the first decade of mobile development, the artistic idea was the approach that many developers chose. Design occurred in an ad hoc manner and was usually conducted as HTML was generated. Design evolved out of an artistic vision that evolved as Web page construction occurred.

Even today, many developers use mobile apps as poster children for "limited design." They argue that immediacy and volatility of the mobile market mitigate against formal design; that design evolves as an application is built (coded), and that relatively little time should be spent on creating a detailed design model. This argument has merit, but only for relatively simple apps. When content and function are complex; when the size of the mobile app encompasses hundreds or thousands of content objects, functions, and analysis classes; and when the success of the app will have a direct impact on the success of the business, design cannot and should not be taken lightly. This reality leads us to Nielsen's second approach—"the engineering ideal of solving a problem for a customer."

13.1 THE CHALLENGES

Although mobile devices have many features in common with each other, their users often have very different perceptions of what features they expect to be bundled in each. Some users expect the same features that are provided on their personal computers. Others focus on the freedom that portable devices give them and gladly accept the reduced functionality in the mobile version of a familiar software product. Still others expect unique experiences not possible on traditional computing or entertainment devices. The user's perception of "goodness" might be more important than any of the technical quality dimensions of the mobile product itself.

13.1.1 Development Considerations

Like all computing devices, mobile platforms are differentiated by the software they deliver—a combination of operating system (e.g., Android or iOS) and a small subset of the hundreds of thousands of mobile software products that provide a very wide range of functionality. New tools allow individuals with little formal training to create and sell apps alongside other apps developed by large teams of software developers.

Even though apps can be developed by amateurs, many software engineers think that MobileApps are among the most challenging software systems being built today [Voa12]. Mobile platforms are very complex. Both the Android and iOS operating systems contain over 12 million lines of code. Mobile devices often have mini browsers that will not display the full set of content available on a Web page. Different mobile devices use different operating systems and platform-dependent development environments. Mobile devices tend to have smaller screen sizes and more varied screen sizes than personal computers. This may require greater attention to user interface design issues, including decisions to limit display of some content. MobileApps

must be designed to take intermittent connectivity outages into account, limitations on battery life, and other device constraints[1] [Whi08].

System components in mobile computing environments are likely to change their locations while their apps are running. To maintain connectivity in nomadic networks,[2] coordination mechanisms for discovering devices, exchanging information, maintaining security and communication integrity, and synchronizing actions must be developed. There is always a trade-off between security and other elements of the mobile product design.

In addition, software engineers must identify the proper design trade-offs between the expressive power of the MobileApp and stakeholder security concerns. Developers must seek to discover algorithms (or adapt existing algorithms) that are energy efficient to conserve battery power when possible. Middleware may have to be created to allow different types of mobile devices to communicate with each other in the same mobile networks [Gru00].

Software engineers should craft a user experience that takes advantage of device characteristics and context-aware applications. The nonfunctional requirements (e.g., security, performance, usability) are a bit different from those for either WebApps or desktop software applications. There is always a trade-off between security and other elements of the mobile design. Testing mobile software products (Chapter 21) provides additional challenges because users expect that they will work in a large number of physically different environments. Portability is another challenge for software engineers as there are several popular device platforms. It is expensive to develop and support applications for multiple device platforms [Was10].

13.1.2 Technical Considerations

The low cost of adding Web capabilities to everyday devices such as phones, cameras, and TVs is transforming the way people access information and use network services [Sch11]. Among the many technical considerations that MobileApps should address are the following:

Multiple hardware and software platforms. It is not at all unusual for a mobile product to run on many different platforms (both mobile and stationary) with a range of differing levels of functionality. The reasons for these differences are in part because the hardware and software available are quite different from device to device. This increases both development cost and time. It also can make configuration management (Chapter 22) more difficult.

Many development frameworks and programming languages. Mobile products are currently being written in several distinct programming or scripting languages (e.g., HTML5, JavaScript, Java, Swift, and C#) for a multitude of popular development frameworks (e.g., Android, iOS, Xamarin, Windows, AngularJS). Very few mobile devices allow direct development on a device itself. Instead, mobile developers typically use emulators running on desktop

1 Available at http://www.devx.com/SpecialReports/Article/37693.

2 *Nomadic networks* have changing connections to mobile devices or servers.

development systems. These emulators may or may not accurately reflect the limitations of the device itself. Thin-client applications are often easier to port to multiple devices than applications designed to run exclusively on the mobile device.

Many app stores with different rules and tools. Each mobile platform has its own app store and its own standards for accepting apps (e.g., Apple,[3] Google,[4] Microsoft,[5] and Amazon[6] publish their own standards). Development of a mobile product for multiple platforms must proceed separately, and each version of the app needs its own standards expert.

Very short development cycles. The marketplace for mobile products is very competitive, and software engineers are likely to make use of agile development processes when building MobileApps in an effort to reduce development time [Was10].

User interface limitations and complexities of interaction with sensors and cameras. Mobile devices have smaller screen sizes than personal computers and a richer set of interaction possibilities (touch, gesture, camera, etc.) and usage scenarios based on context awareness. The style and appearance of the user interface is often dictated by the nature of platform-specific development tools [Rot02]. Allowing smart devices to interact with smart spaces offers the potential to create personalized, networked, high-fidelity application platforms such as those seen by merging smartphones and car infotainment systems.[7]

Effective use of context. Users expect MobileApps to deliver personalized user experiences based on the physical location of a device in relation to the available network features. User interface design and context-aware applications are discussed in greater detail in Section 13.4.

Power management. Battery life is often one of the most limiting constraints on many mobile devices. Backlighting, reading and writing to memory, using wireless connections, making use of specialized hardware, and processor speed all impact power usage and need to be considered by software developers [Mei09].

Security and privacy models and policies. Wireless communication is difficult to protect from eavesdropping. Preventing *man-in-the-middle-attacks*[8] in automotive applications can be critical to the safety of the users [Bos11]. Data stored on a mobile device are subject to theft if a device is lost or a malicious app is downloaded. Software policies that increase the level of confidence in the security and privacy of a MobileApp often reduce the usability of the app and the spontaneity of the communication among users [Rot02].

3 https://developer.apple.com/appstore/guidelines.html.
4 http://developer.android.com/distribute/googleplay/publish/preparing.html.
5 http://msdn.microsoft.com/en-us/library/ff941089%28v=vs.92%29.aspx.
6 https://developer.amazon.com/apps-and-games/app-submission/android.
7 When used in an automotive setting, smart devices should be able to restrict access to services that may distract the driver and allow hands-free operation when a vehicle is moving [Bos11].
8 These attacks involve a third party intercepting communications between two trusted sources and impersonating one or both of the parties.